Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la carrera:

Ingeniería en Informática.

Nombre de la asignatura:

Programación cliente - servidor.

Semestre y grupo:

7AI

Actividad:

Implementar CRUD con relaciones, validaciones, errores consistentes y ORM.

Nombre del alumno:

Aaron Jesús Eloy Aguilar León.

Chi Chablé Miguel Eduardo.

Ek Noh Edgar Ernesto.

Nombre del docente:

Jenner Noe Che Méndez.

Lugar y fecha:

San Francisco de Campeche, Campeche a 02 de Noviembre de 2026.

1.- Introducción y la arquitectura de la actividad.

En esta actividad se implementa un backend robusto y modular desarrollado en FastAPI bajo una arquitectura orientada a servicios REST.

Es por esa misma razón que se utiliza un mapeo objeto-relacional (ORM SQLAlchemy) conectado a una base de datos local optimizada (SQLite), cumpliendo con los estándares de validación de datos mediante Pydantic y asegurando la persistencia transaccional de los equipos y préstamos en el entorno de la bitácora agropecuaria.

2.- Modelo Relacional (Usuario - Préstamo - Equipo).

El sistema se compone de tres entidades principales interrelacionadas para gestionar de forma íntegra el flujo operativo:

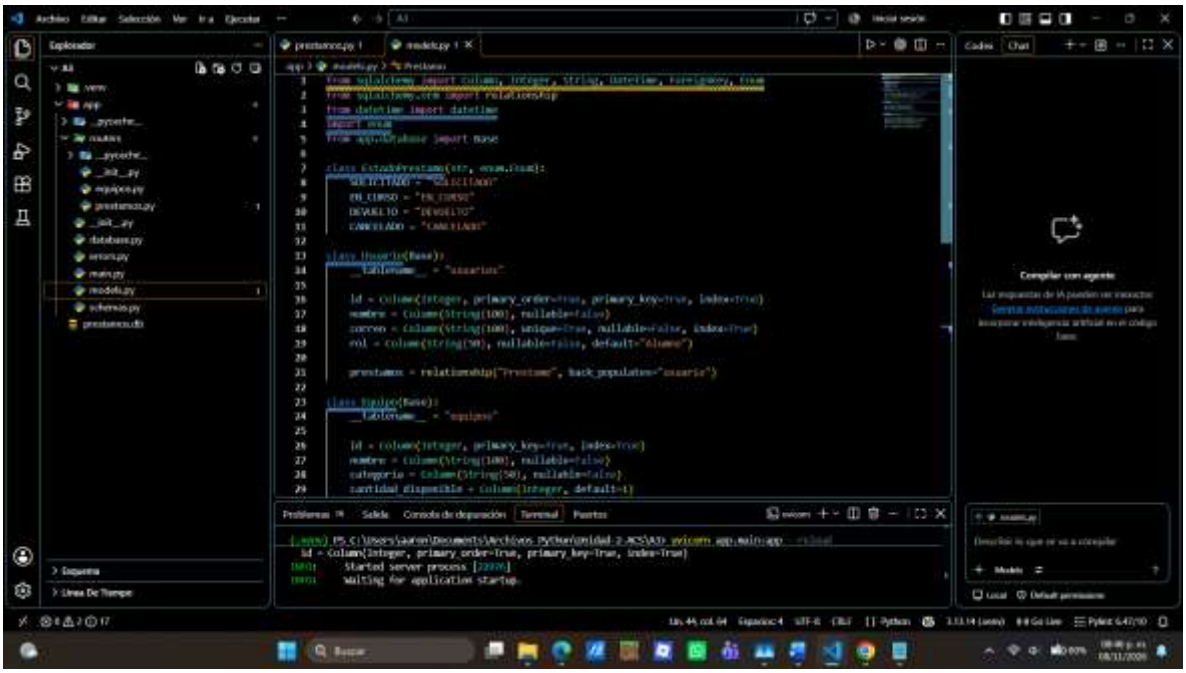
Usuario: Almacena la información de los solicitantes (ID, nombre, correo electrónico validado y rol).

Equipo: Catálogo de herramientas e instrumentos agrícolas (ID, nombre, categoría y cantidad disponible en stock).

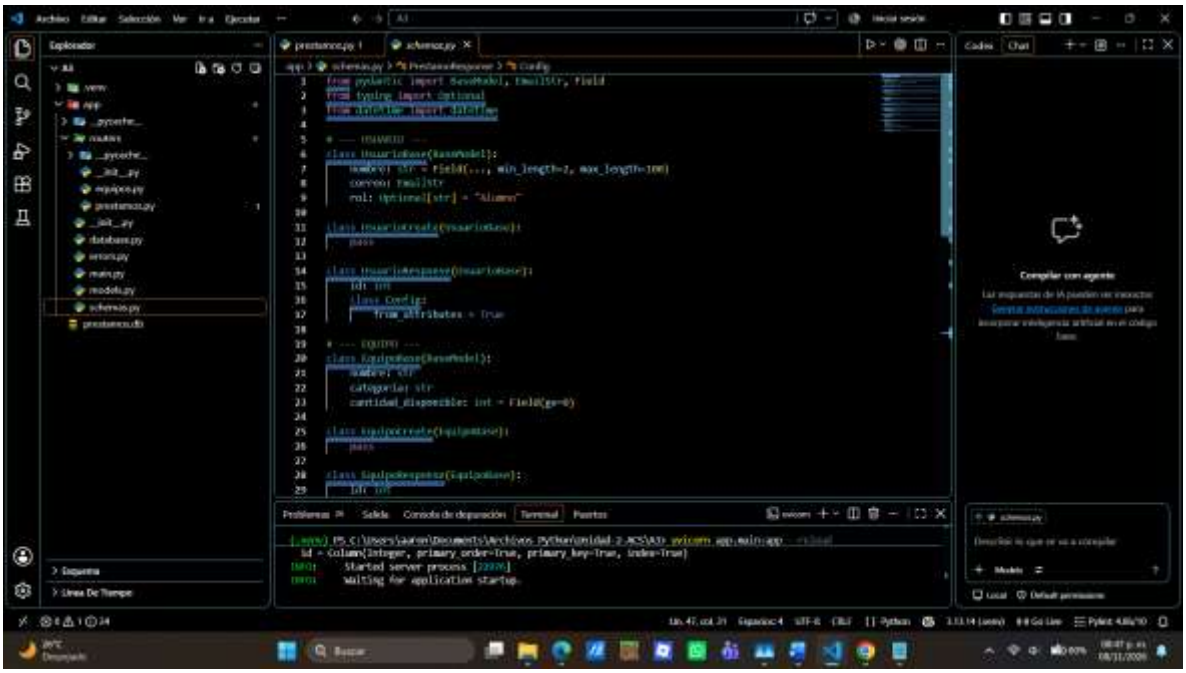
Préstamo: Tabla intermedia transaccional que registra la relación de asignación (ID de préstamo, ID de usuario, ID de equipo, fecha de solicitud, fecha de devolución y estado actual del préstamo).

3.- Códigos de Modelo ORM y Schemas de Validación.

Modelo ORM.



Schemas de Validación.



4.- Implementación de CRUD y Reglas de Negocio.

El sistema implementa las siguientes operaciones y restricciones lógicas:

- Control de Stock Automatizado: Al registrar un préstamo (POST /prestamos/), el sistema valida que el equipo exista y posea una cantidad_disponible > 0. De cumplirse, descuenta automáticamente una unidad del inventario.
- Devolución y Reposición: Al procesar la devolución de un equipo (PUT /prestamos/{id}/devolver), el estado cambia a "DEVUELTO", se registra la fecha actual y se reponen las existencias en el inventario del equipo correspondiente.

5.- Manejo Uniforme de Errores y Transacciones.

Se implementaron excepciones personalizadas mediante HTTPException para estandarizar las respuestas ante fallos comunes:

400 Bad Request: Lanzado cuando se intenta solicitar un equipo sin stock disponible o un préstamo ya devuelto.

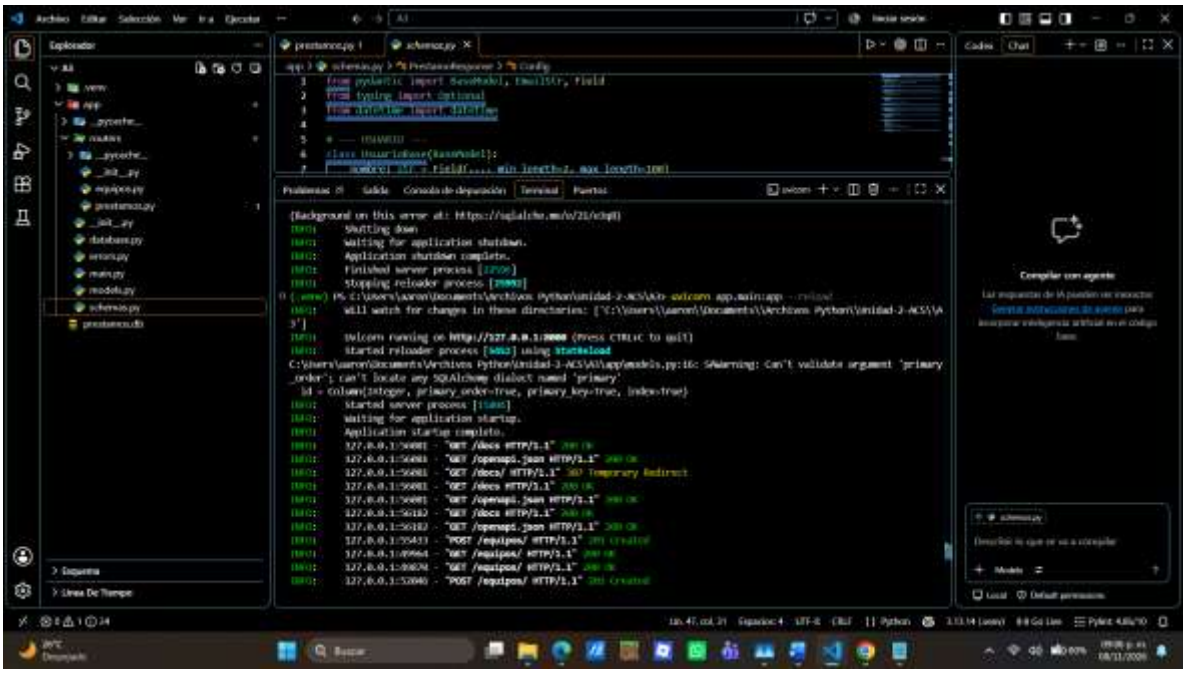
404 Not Found: Lanzado cuando no se localiza un equipo o folio de préstamo en la base de datos.

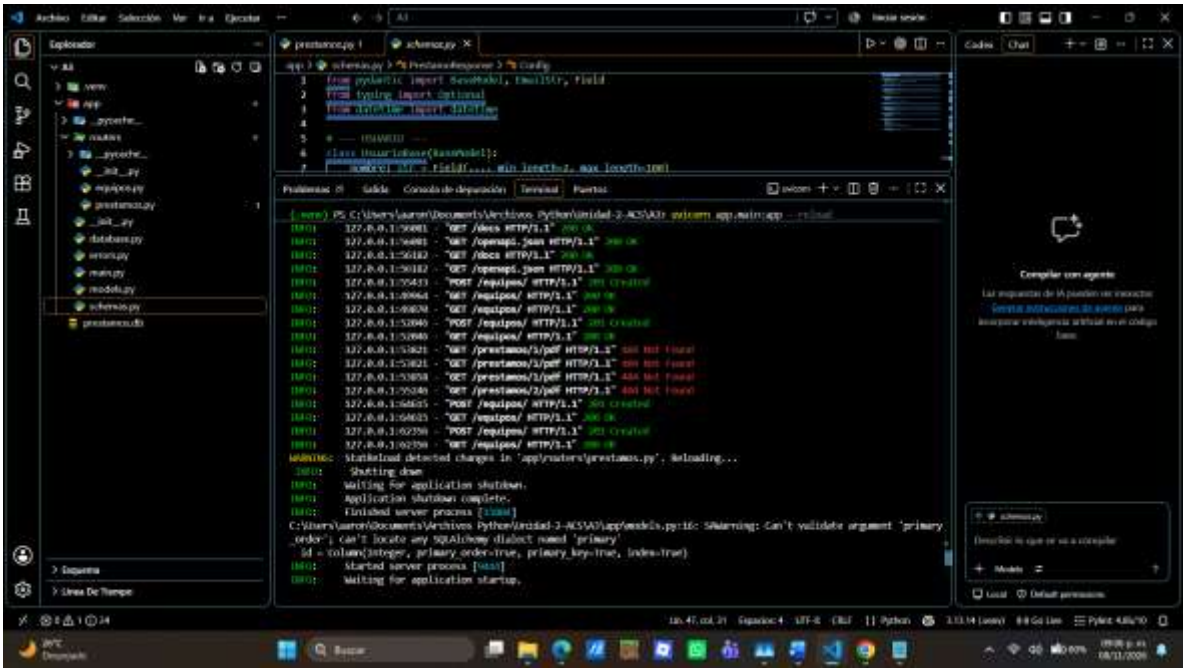
Control transaccional mediante db.commit() y db.rollback() para garantizar la atomicidad de las operaciones de stock.

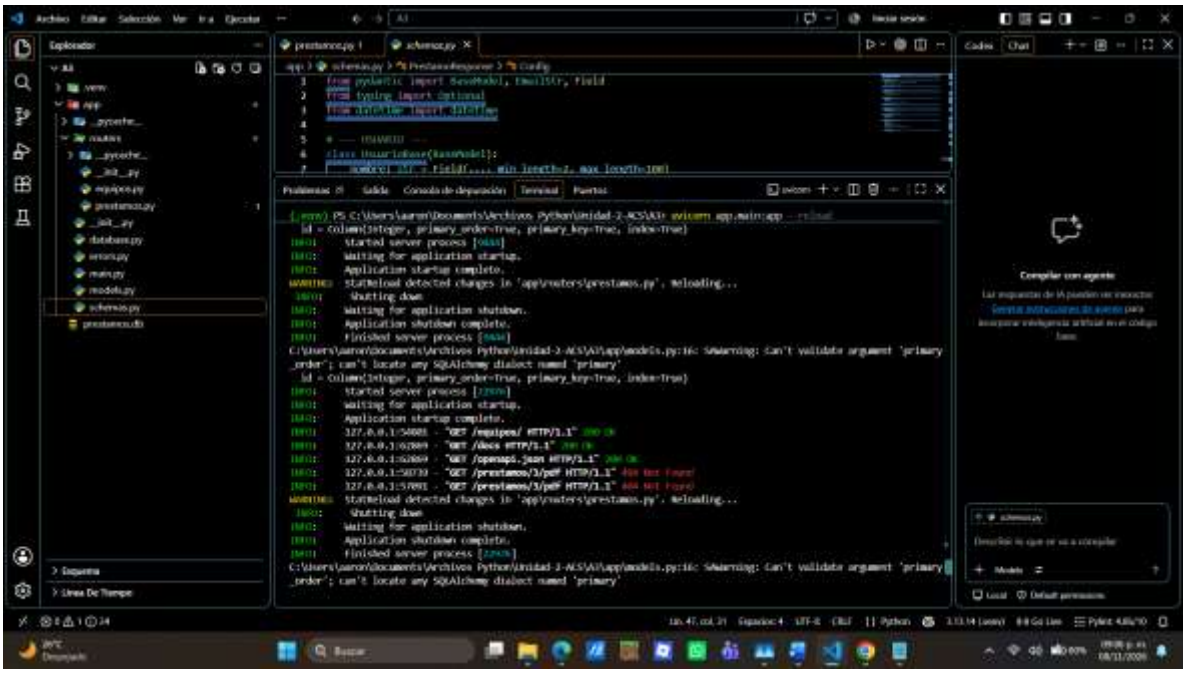
6.- Pruebas de Casos Exitosos y Fallidos (Evidencias OpenAPI / Swagger)

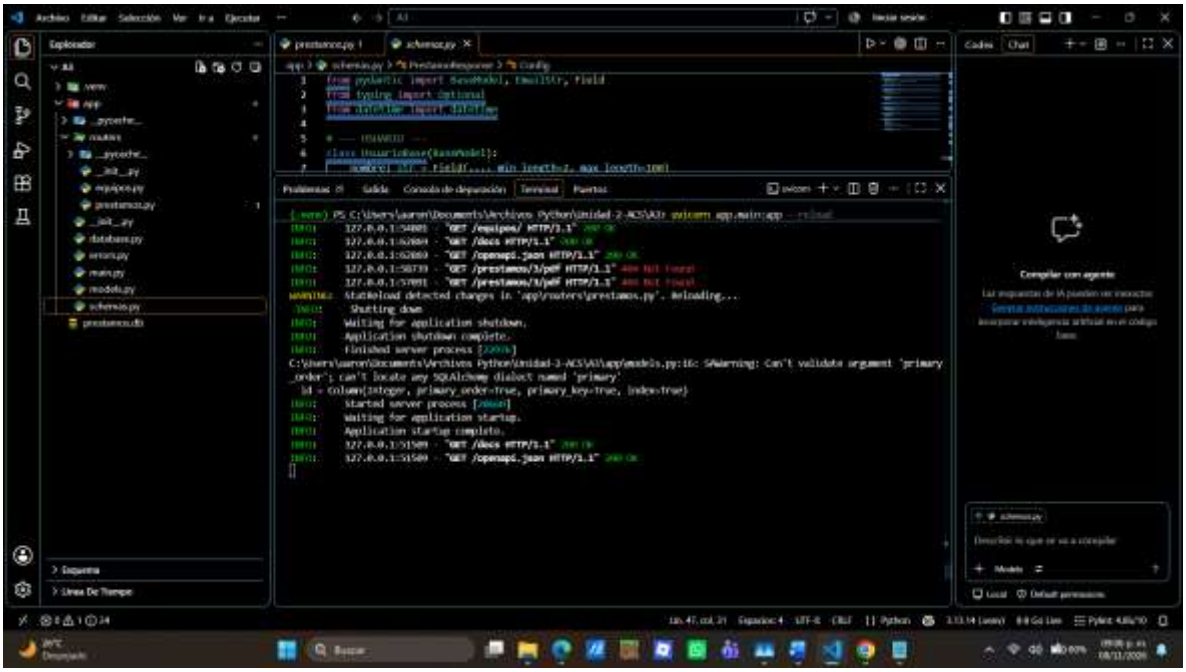
Caso Exitoso: Alta de equipo con stock positivo y registro de préstamo correspondiente con código de respuesta 201 Created.

Caso Fallido: Intento de préstamo sobre un equipo con inventario en 0, respondiendo con código 400 Bad Request y el mensaje detallado "Equipo no disponible para préstamo".

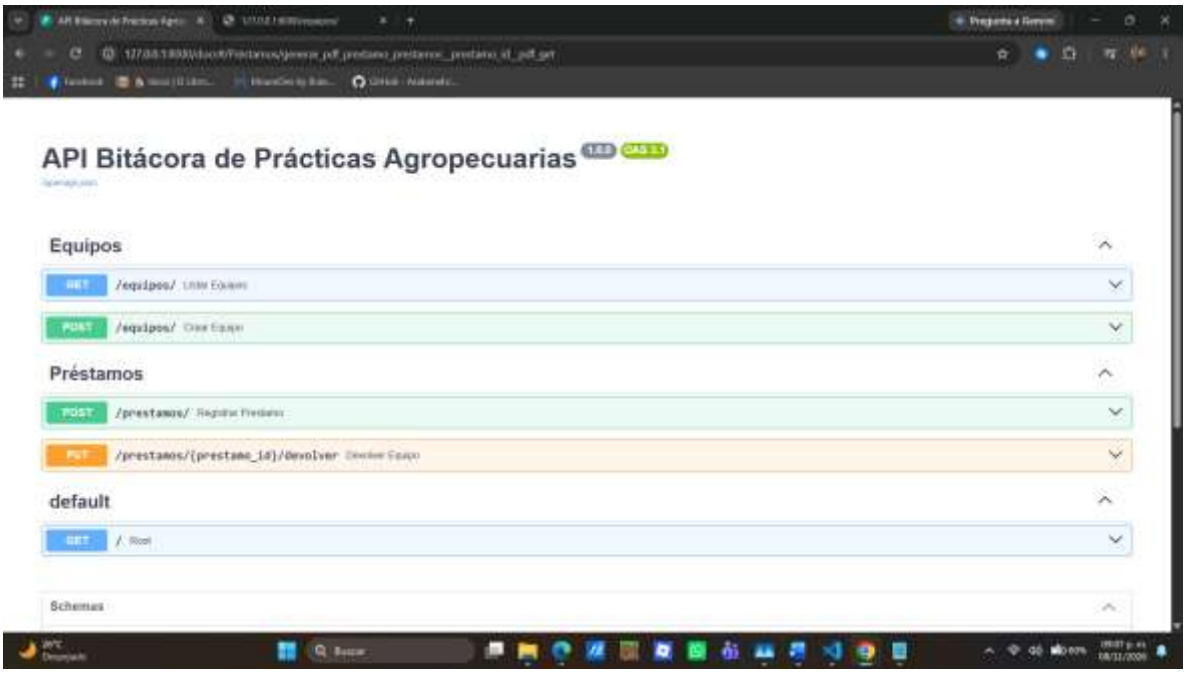


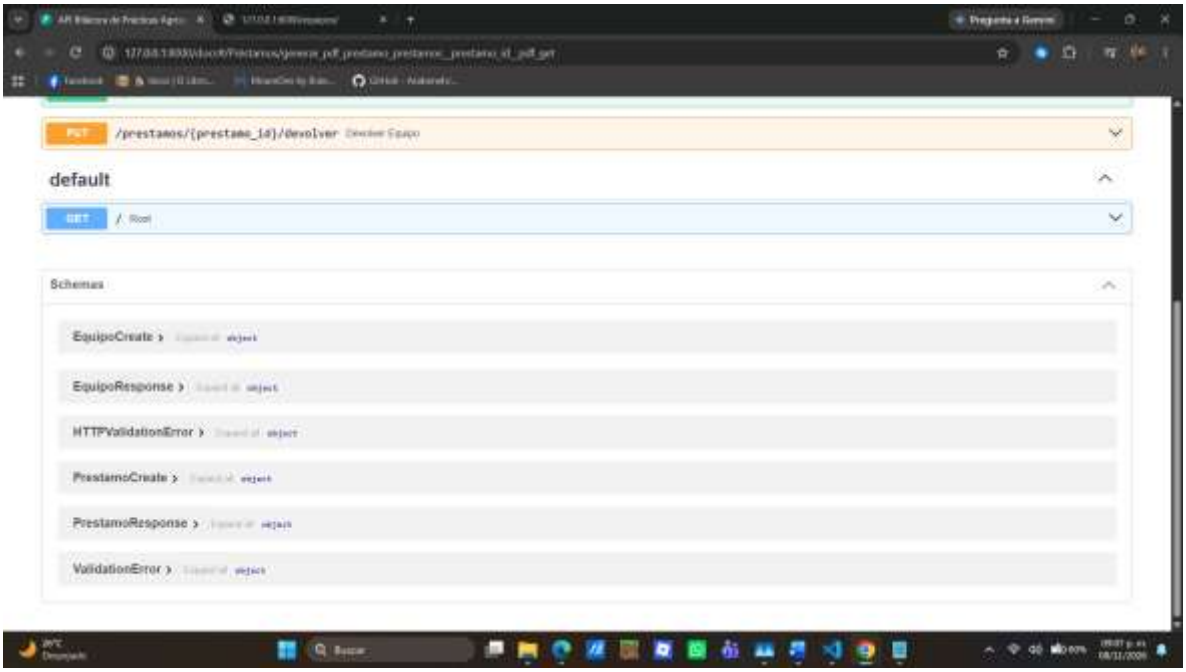






Anexo de la interfaz interactiva.

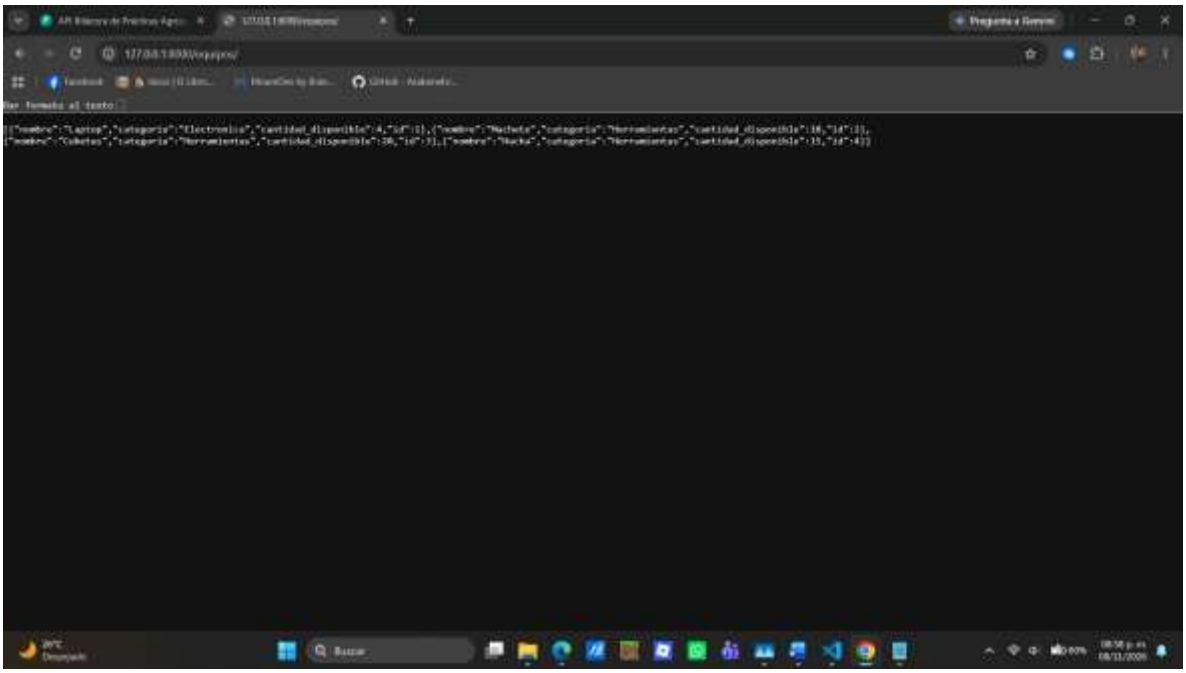




Bajo qué dirección y puerto se implementó de forma local.

http://127.0.0.1:8000/equipos/

Evidencia de registro.



Anexo vínculo de los archivos que se usaron, ya que github no permitió la subida por el tamaño máximo de archivo 25mb máximo.

A3.rar